

WEEK 4 – CYBER SECURITY INTERNSHIP

Network Configuration, Connectivity & Basic Security Analysis

Practical Evidence Report

Prepared by: Amar Pratap

Organization: DG Interns Hub

Environment: VMware Workstation + Kali Linux

All evidence in this report is taken from the submitted laboratory screenshots. Values that are obscured in the screenshots are intentionally not reconstructed or invented.

TABLE OF CONTENTS

Week 4 – Cyber Security Internship | Amar Pratap | DG Interns Hub

Section	Page
1. Objective & Lab Overview	3
2. Evidence Map	3
3. VMware Network Configuration	4
4. IP Address & Interface Configuration	5–6
5. Connectivity & Routing Tests	7–8
6. DNS Resolution & Socket Inspection	9
7. Nmap Scanning & Result Preservation	10–11
8. Traceroute & Web/TLS Inspection	12–13
9. System Identification & Running Services	14
10. Wireshark Packet Analysis	15
11. Findings & Security Observations	16
12. Conclusion & Submission Checklist	17

Submission / Evidence Status

Task 1 – Network setup	✓ Evidenced	VMware NAT + interface/IP + connectivity screenshots
Task 2 – Web server	■ Not evidenced	No target Apache/Nginx status or target web-page screenshot was submitted
Task 3 – Nmap	✓ Evidenced	Nmap target scan + saved nmap_scan.txt
Task 4 – Wireshark	■ Partial	ICMP capture is evidenced; DNS/HTTP packet captures are not included
Task 5 – Full workflow	✓/■	Core workflow documented; two evidence gaps above remain

Before final submission: add one screenshot proving the target Apache/Nginx service is running and, if required by your instructor, add Wireshark captures filtered with **dns** and **http**. These are not fabricated in this report.

1. Objective & Lab Overview

The purpose of this Week 4 practical was to configure and verify a virtual Linux networking environment and perform basic network/security observation using standard command-line and packet-analysis tools.

The captured evidence covers VMware network configuration, interface/IP inspection, routing, connectivity testing, DNS resolution, socket inspection, Nmap scanning, traceroute, HTTP/TLS request inspection, operating-system identification, running services and Wireshark packet capture.

The screenshots show a Kali Linux environment running in VMware Workstation. The VMware Network Adapter is configured in NAT mode and the Linux interface is operational.

2. Evidence Map

Evidence	Practical	Screenshot
1	VMware Network Adapter / NAT	Screenshot 1
2	IP address configuration	Screenshots 2 & 4
3	Internet connectivity	Screenshot 3
4	Routing	Screenshot 5
5	Private-network connectivity	Screenshot 6
6	DNS resolution	Screenshot 7
7	Interface/link state	Screenshot 8
8	Socket inspection	Screenshot 9
9	Nmap scanning	Screenshots 10, 11 & 17
10	Traceroute	Screenshot 12
11	HTTP/TLS request analysis	Screenshots 13 & 14
12	OS identification	Screenshot 15
13	Running services	Screenshot 16
14	Wireshark ICMP capture	Screenshot 18

3. VMware Network Configuration

The VMware virtual machine's Network Adapter is shown as connected and configured for **NAT**. NAT allows the virtual machine to share the host's network connection while remaining behind the virtualization layer.

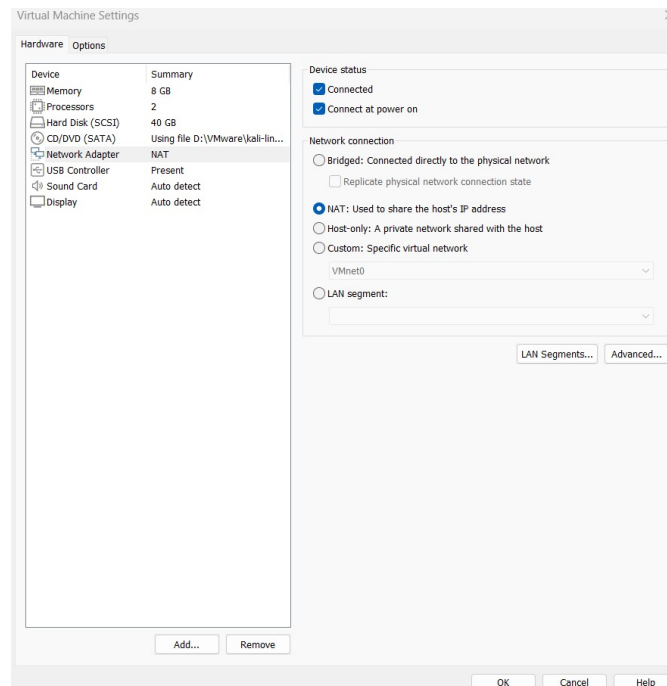


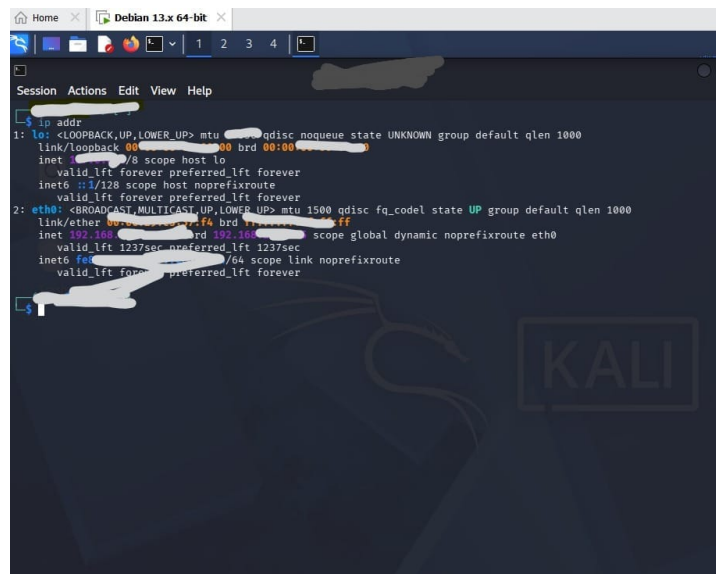
Figure 1. VMware Network Adapter configured in NAT mode.

Observation

The adapter is marked Connected and Connect at power on is enabled. This establishes the virtual networking path used by the Kali VM.

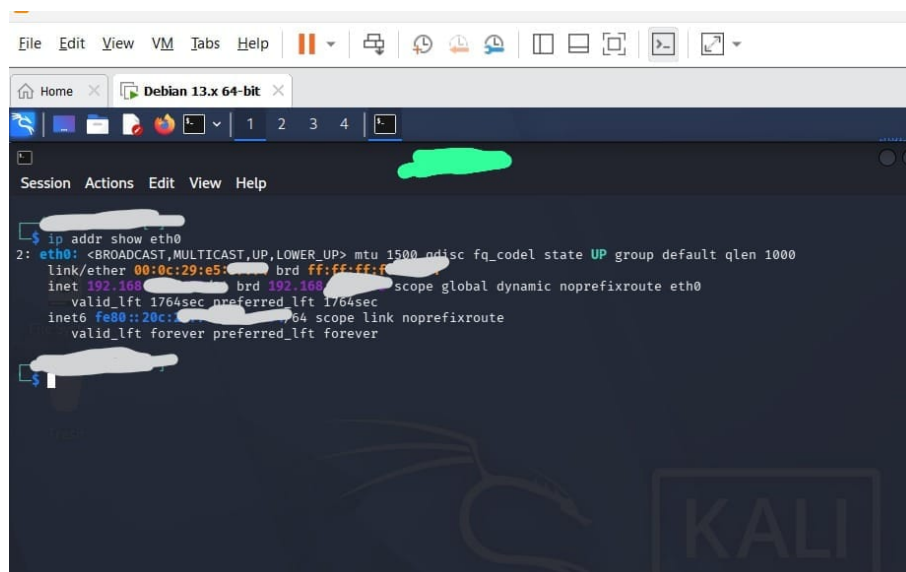
4. IP Address & Interface Configuration

The `ip addr` and `ip addr show eth0` outputs were used to verify that the Ethernet interface is up and has an IPv4 address. The screenshots show the interface state as UP and an address in the 192.168.x.x private range.



```
Session Actions Edit View Help
$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:e5:14:14 brd ff:ff:ff:ff:ff:ff
    inet 192.168.1.100/24 scope global dynamic noprefixroute eth0
        valid_lft 1237sec preferred_lft 1237sec
    inet6 fe80::20c:29e5:1414:1414/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
$
```

Figure 2. `ip addr` output showing the active Ethernet interface and IPv4 configuration.



```
File Edit View VM Tabs Help
Session Actions Edit View Help
$ ip addr show eth0
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:e5:14:14 brd ff:ff:ff:ff:ff:ff
    inet 192.168.1.100/24 scope global dynamic noprefixroute eth0
        valid_lft 1764sec preferred_lft 1764sec
    inet6 fe80::20c:29e5:1414:1414/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
$
```

Figure 3. `ip addr show eth0` used for focused interface verification.

Additional interface verification

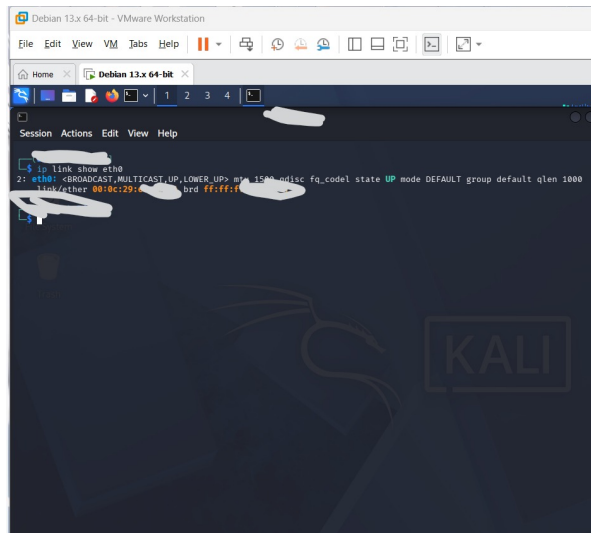
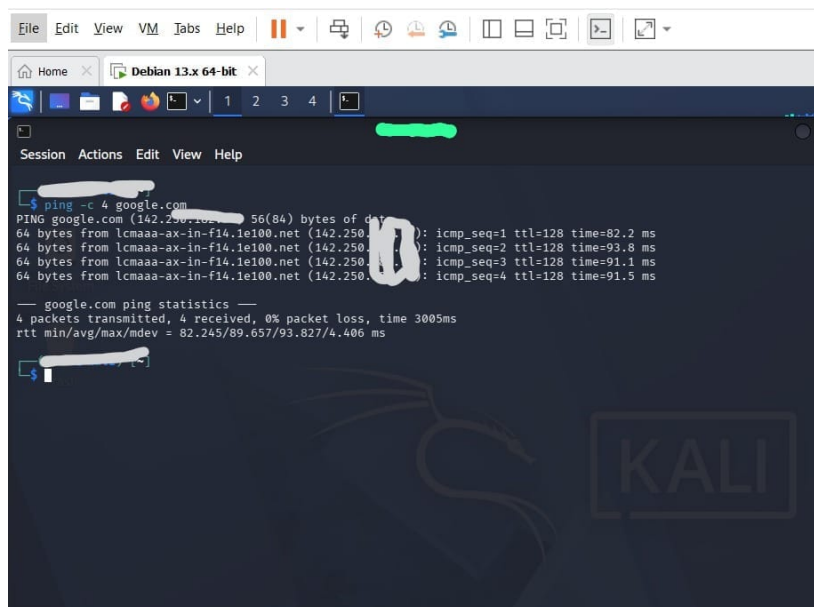


Figure 4. ip link show eth0 confirms the Ethernet link is UP.

5. Connectivity & Routing Tests

Internet connectivity was tested with ping. The submitted result shows four packets transmitted, four received and 0% packet loss for the Google connectivity test.

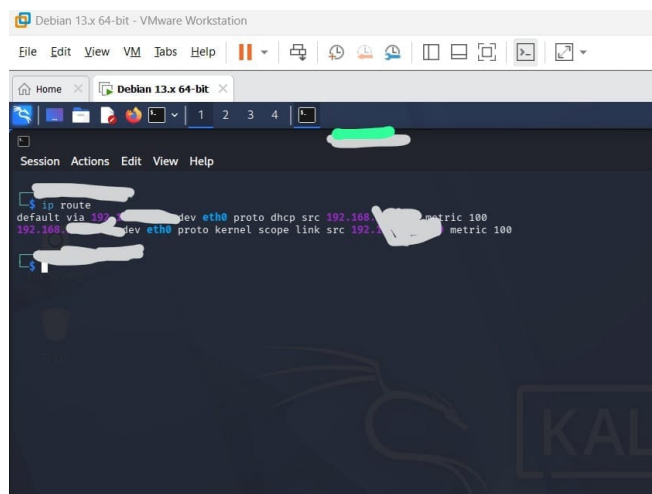


```
File Edit View VM Tabs Help
Debian 13.x 64-bit x
Session Actions Edit View Help
$ ping -c 4 google.com
PING google.com (142.250.104.100) 56(84) bytes of data:
64 bytes from lcmaaa-ax-in-f14.1e100.net (142.250.104.100): icmp_seq=1 ttl=128 time=82.2 ms
64 bytes from lcmaaa-ax-in-f14.1e100.net (142.250.104.100): icmp_seq=2 ttl=128 time=93.8 ms
64 bytes from lcmaaa-ax-in-f14.1e100.net (142.250.104.100): icmp_seq=3 ttl=128 time=91.1 ms
64 bytes from lcmaaa-ax-in-f14.1e100.net (142.250.104.100): icmp_seq=4 ttl=128 time=91.5 ms

--- google.com ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3005ms
rtt min/avg/max/mdev = 82.245/89.657/93.827/4.406 ms
```

Figure 5. ping -c 4 google.com: 4/4 packets received and 0% packet loss.

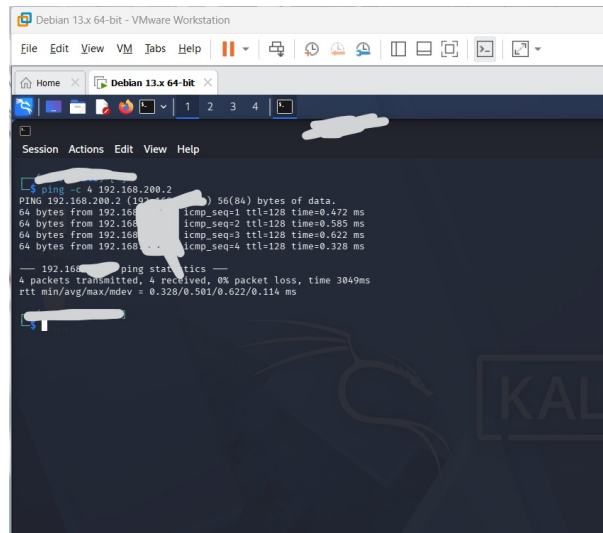
The routing table was also inspected using **ip route**. The screenshot shows a default route through the private gateway and the eth0 interface.



```
Debian 13.x 64-bit - VMware Workstation
File Edit View VM Tabs Help
Debian 13.x 64-bit x
Session Actions Edit View Help
$ ip route
default via 192.168.1.1 dev eth0 proto dhcp src 192.168.1.100 metric 100
192.168.1.0/24 dev eth0 proto kernel scope link src 192.168.1.100 metric 100
```

Figure 6. ip route output showing the default route and eth0 interface.

A separate ping test was performed against a 192.168.200.x private address and returned four replies with 0% packet loss, demonstrating reachability to that private-network destination.



```
Debian 13.x 64-bit - VMware Workstation
File Edit View VM Tabs Help
Home x Debian 13.x 64-bit x
Session Actions Edit View Help
ping -c 4 192.168.200.2
PING 192.168.200.2 (192.168.200.2) 56(84) bytes of data:
64 bytes from 192.168.200.2: icmp_seq=1 ttl=128 time=0.472 ms
64 bytes from 192.168.200.2: icmp_seq=2 ttl=128 time=0.585 ms
64 bytes from 192.168.200.2: icmp_seq=3 ttl=128 time=0.622 ms
64 bytes from 192.168.200.2: icmp_seq=4 ttl=128 time=0.328 ms
--- 192.168.200.2 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3049ms
rtt min/avg/max/mdev = 0.328/0.581/0.622/0.116 ms
```

Figure 7. Private-network ping test with 4 packets received and 0% packet loss.

6. DNS Resolution & Socket Inspection

The **nslookup** command was used to resolve google.com. The screenshot shows a configured DNS server and returned IPv4/IPv6 address records.

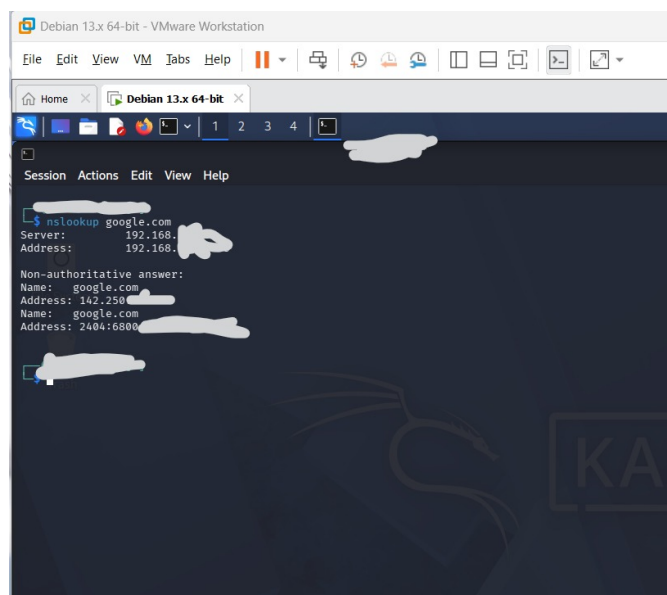


Figure 8. nslookup google.com showing successful DNS resolution.

The **ss -tuln** command was used to inspect listening TCP/UDP sockets. The submitted output shows the column structure and no visible listening entries in the captured view.

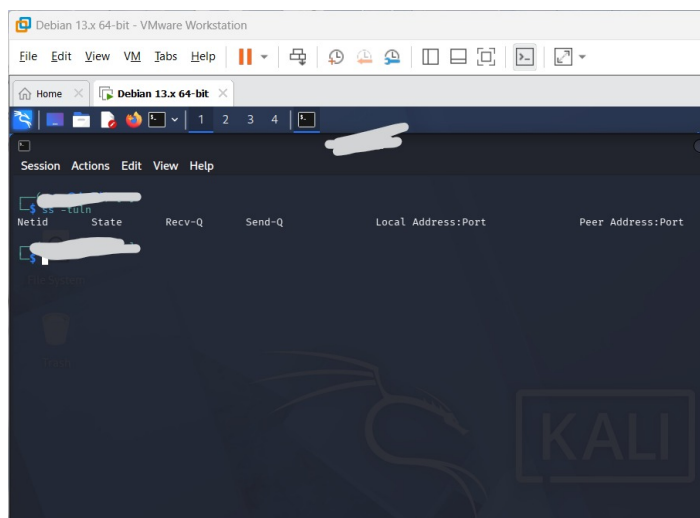


Figure 9. ss -tuln socket inspection.

7. Nmap Scanning & Result Preservation

Nmap was used for service/version discovery against authorized targets. The submitted screenshots show two relevant outcomes: a localhost scan and a private-network target scan.

The private target scan reports the host as up, while the 1000 default TCP ports are shown as filtered/no-response. Therefore, the evidence does **not** support claiming that an open TCP service was discovered on that target.

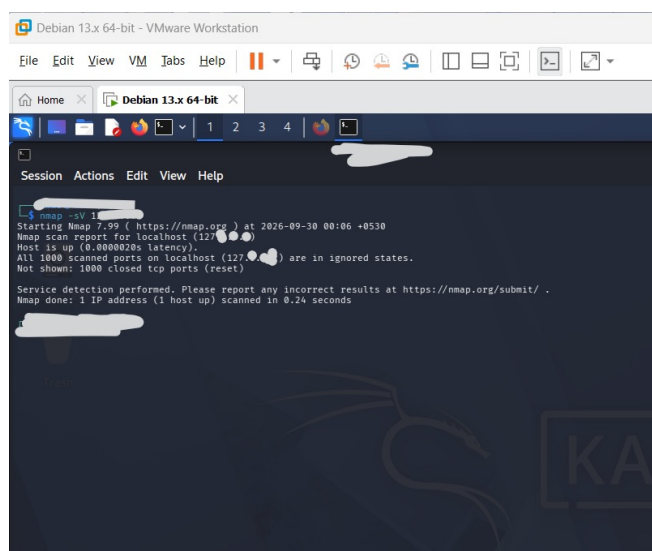


Figure 10. Nmap -sV scan of localhost; the visible output reports the host and filtered ports.

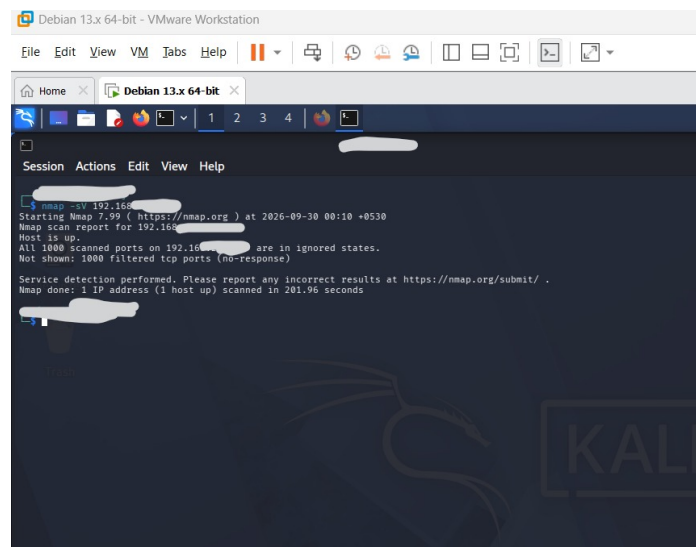
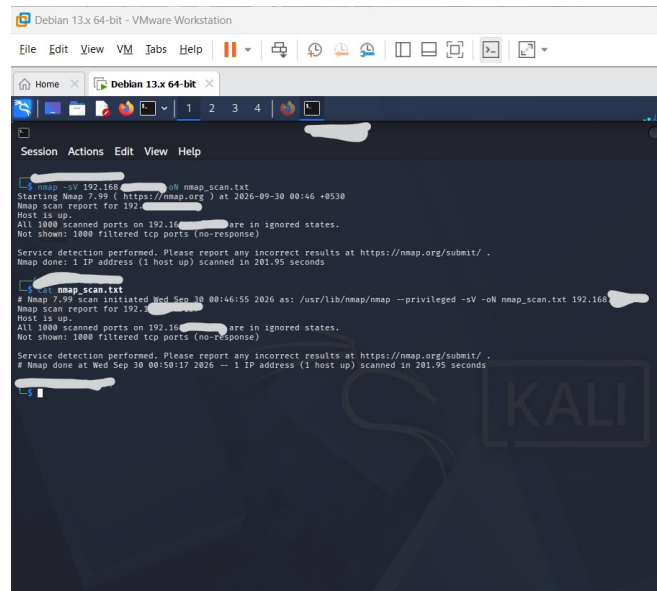


Figure 11. Nmap -sV scan of the 192.168.x.x target; the visible result shows 1000 TCP ports filtered/no-response.

The result was saved to `nmap_scan.txt` using the `-oN` option. The saved file was then displayed with `cat`, providing reproducible evidence of the scan command and output.



The screenshot shows a terminal window titled "Debian 13.x 64-bit" within a VMware Workstation environment. The terminal displays the output of an Nmap scan saved to a file named "nmap_scan.txt". The output is as follows:

```
-- nmap -sV 192.168.1.100 -o nmap_scan.txt
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-30 08:46 +0530
Nmap scan report for 192.168.1.100
Host is up.
All 1000 scanned ports on 192.168.1.100 are in ignored states.
Not shown: 1000 filtered tcp ports (no-response)
Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 201.95 seconds

cat nmap_scan.txt
# Nmap 7.99 scan initiated Wed Sep 30 08:46:55 2026 as: /usr/lib/nmap/nmap --privileged -sV -o nmap_scan.txt 192.168.1.100
Nmap scan report for 192.168.1.100
Host is up.
All 1000 scanned ports on 192.168.1.100 are in ignored states.
Not shown: 1000 filtered tcp ports (no-response)
Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
# Nmap done at Wed Sep 30 08:50:17 2026 -- 1 IP address (1 host up) scanned in 201.95 seconds
```

Figure 12. Nmap output saved as nmap_scan.txt and displayed with cat.

8. Traceroute & Web/TLS Inspection

Traceroute was used to observe the path toward google.com. The first visible hop is the private gateway; subsequent hops are represented by * entries in the submitted capture, which can occur when intermediate devices do not return the expected traceroute responses.

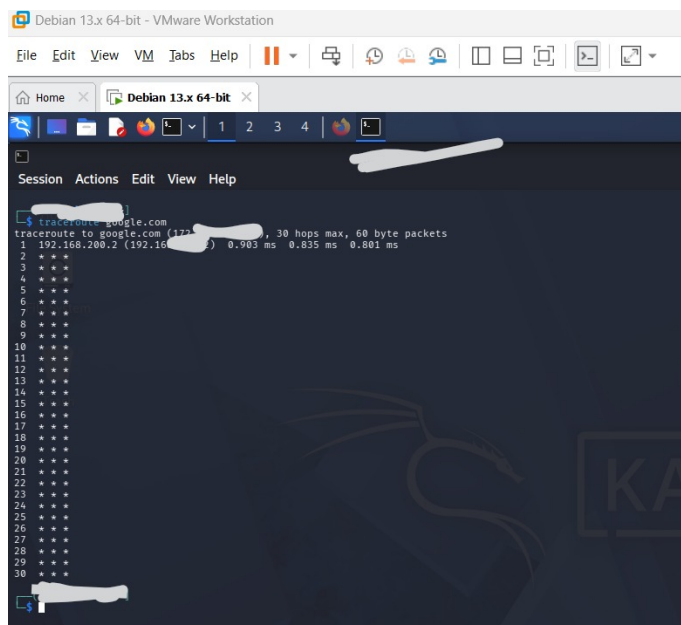


Figure 13. traceroute google.com showing the first private-network hop and non-responsive subsequent hops.

HTTP/TLS behavior was inspected with curl. The header request to example.com returned HTTP/2 200, while the verbose request shows TLS 1.3 negotiation, certificate information and an HTTP/2 response.

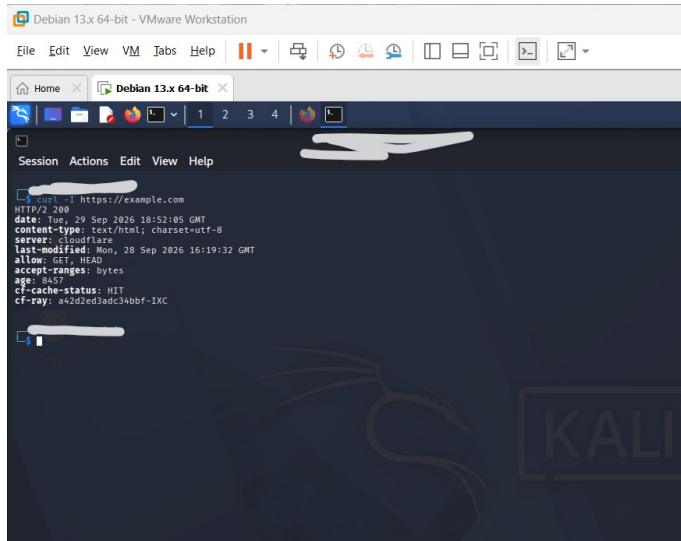


Figure 14. curl -I https://example.com showing HTTP response headers and HTTP/2 200.

9. System Identification & Running Services

The **uname -a** command was used to record kernel/system information. The submitted output identifies the environment as a Kali Linux kernel running on x86_64 architecture.

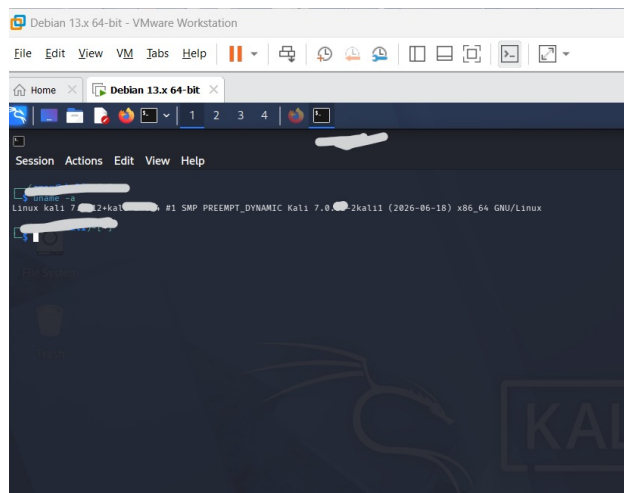


Figure 16. **uname -a** output identifying the Kali Linux kernel environment.

The **systemctl --type=service --state=running** command was used to list active services. The captured output includes services such as NetworkManager, systemd-logind, systemd-timesyncd, udisks2 and VMware-related open-vm-tools.

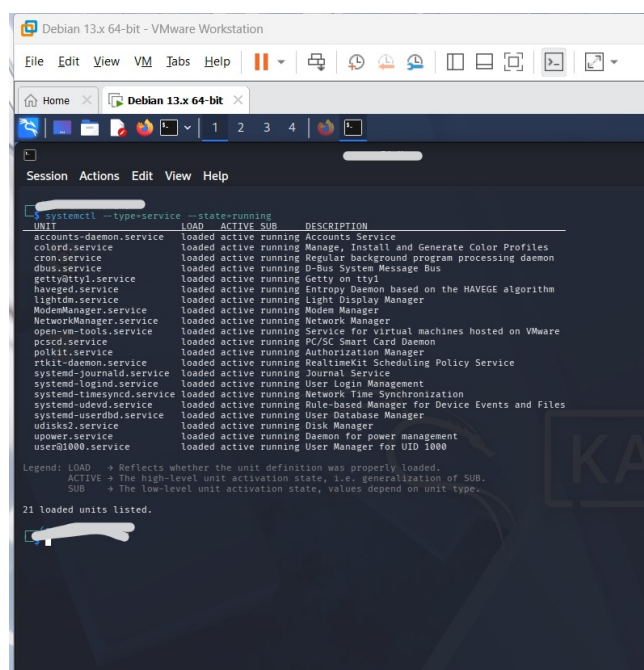


Figure 17. **systemctl** output showing active/running services.

10. Wireshark Packet Analysis

Wireshark was used to capture ICMP traffic generated by a ping test. The display filter **icmp** is visible in the capture. The packet list contains Echo (ping) request and Echo (ping) reply frames between the private-network endpoints.

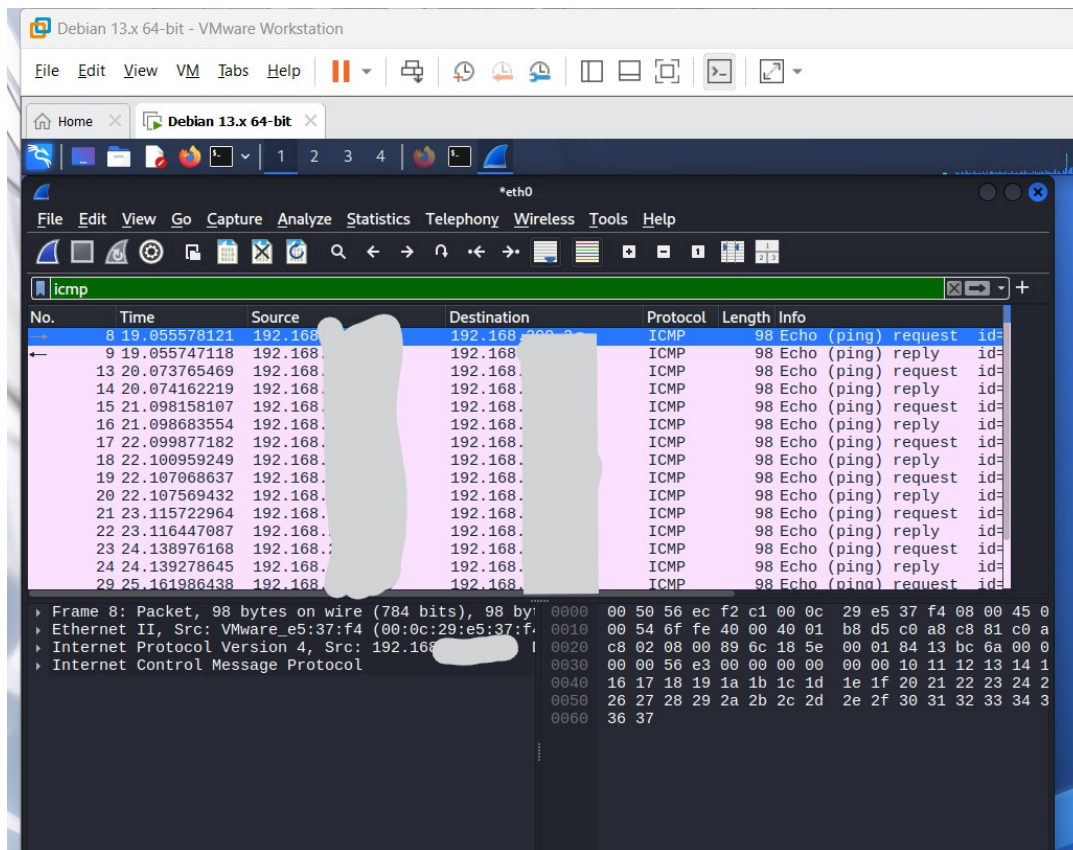


Figure 18. Wireshark ICMP capture showing Echo request and Echo reply packets.

Packet-level observation

The capture demonstrates the request/reply pattern of ICMP: one endpoint sends an Echo request and the destination responds with an Echo reply. This provides packet-level evidence corresponding to the successful ping tests.

11. Findings & Security Observations

- The VMware adapter is connected in NAT mode, providing network access through the host.
- The Kali Ethernet interface is operational and has a private IPv4 configuration.
- Internet connectivity was verified with 0% packet loss in the submitted Google ping test.
- A private 192.168.200.x destination was also reachable with 0% packet loss in the submitted ping test.
- DNS resolution for google.com succeeded.
- The Nmap target scan did not reveal an open TCP service in the default 1000-port set; the visible result reports those ports as filtered/no-response.
- The Nmap scan was preserved in nmap_scan.txt, improving reproducibility of the evidence.
- Traceroute showed the private gateway as the first hop; later hops did not respond in the capture.
- curl demonstrated successful HTTPS communication with example.com and TLS 1.3 negotiation.
- Wireshark provided packet-level ICMP evidence through Echo request/reply frames.

Security considerations

- Use NAT or an isolated/private network when practicing security tools on virtual machines.
- Only scan systems and addresses for which you have authorization.
- Filtered ports indicate that traffic may be blocked or silently dropped; they should not be interpreted as proof that a service is vulnerable.
- Keep unnecessary services disabled and use host/network firewalls to reduce exposure.
- Use encrypted protocols such as HTTPS when transmitting sensitive information.
- Preserve scan and packet-capture evidence so observations can be independently reviewed.

12. Conclusion & Submission Checklist

The Week 4 practical successfully documents a complete basic network-observation workflow: virtual network configuration, interface/IP verification, connectivity testing, routing, DNS resolution, socket inspection, Nmap scanning, traceroute, HTTPS/TLS inspection, system/service identification and Wireshark packet analysis.

The evidence also demonstrates an important security principle: conclusions should be based on the actual output. In particular, the Nmap target screenshot shows filtered/no-response ports rather than open services, so the report records that result without inventing an exposed service.

Final evidence checklist

- VMware NAT configuration
- IP/interface configuration
- Internet ping and private-network ping
- Routing and DNS evidence
- ss socket inspection
- Nmap scan + saved output
- Traceroute evidence
- curl HTTP/TLS evidence
- uname and systemctl evidence
- Wireshark ICMP evidence
- PPT presentation